MegaBankSystem

Симулятор банковской очереди на Java — от талона до окна обслуживания. Изучаем шаблоны очередей, enum-типы и управление потоком клиентов в консольном приложении.

Made with GAMMA

АРХИТЕКТУРА

Структура приложения

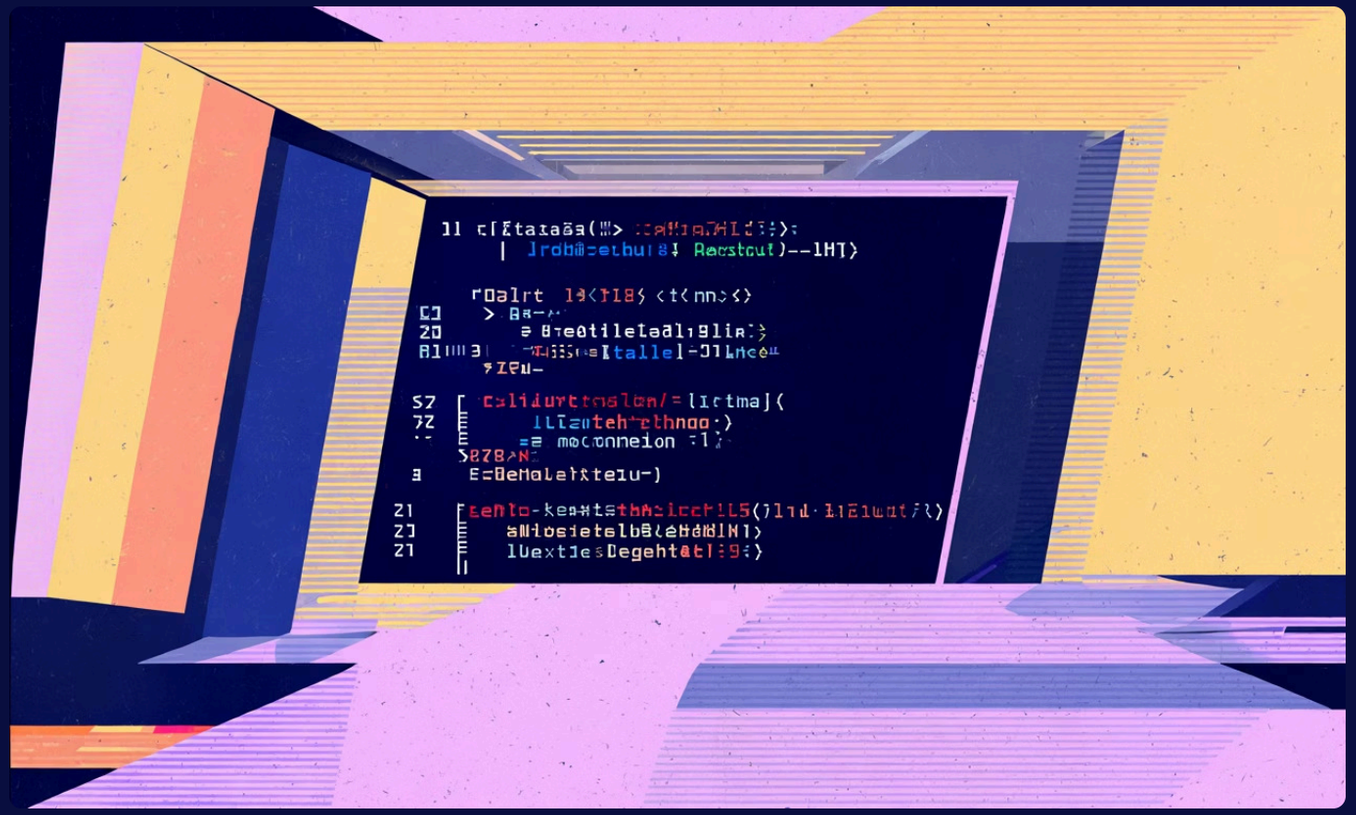
Система построена вокруг трёх ключевых компонентов, взаимодействующих через единую очередь. Клиент получает талон → талон попадает в очередь → сотрудник вызывает следующего клиента к свободному окну.

Все операции выполняются в едином потоке через консольное меню. Общая очередь `commonQueue` — центральный узел, связывающий все компоненты системы.

Made with GAMMA

ТИПЫ ДАННЫХ

Enum Service — строго типизированные услуги



Перечисление `Service` задаёт фиксированный набор банковских услуг с префиксами для нумерации талонов:

Использование `EnumMap` в статистике гарантирует порядок итерации и типобезопасность.

CASH — «К»

Касса

LOAN — «КР»

Кредитование

CARD — «КРТ»

Выпуск карт

VIP — «VIP»

Премиум

Made with GAMMA

ЯДРО СИСТЕМЫ

Класс Ticket — талон клиента

```
static class Ticket {
    String number;    // "K-472", "VIP-831"
    Service service;  // тип услуги

    Ticket(Service s, int id) {
        this.service = s;
        this.number = s.prefix + "-" + id;
    }
}
```

Как формируется номер?

Префикс услуги + случайный ID (100–999):

- CASH → **K-472**
- VIP → **VIP-831**
- LOAN → **KP-205**

Random.nextInt(899) + 100 генерирует уникальный трёхзначный номер для каждого талона.

Made with GAMMA

СТРУКТУРЫ ДАННЫХ

Очередь на основе LinkedList

Почему LinkedList?

Интерфейс `Queue<Ticket>` реализован через `LinkedList` — стандартный выбор для FIFO-очередей в Java.

add(t)

Добавление в конец очереди — **O(1)**

poll()

Извлечение из головы — **O(1)**

isEmpty()

Проверка на пустоту — **O(1)**

Made with GAMMA

Выдача талона: takeTicket()

1

Клиент выбирает услугу

Нажатие 1–4 в меню терминала

2

Генерация номера

Random.nextInt(899) + 100 → уникальный ID

3

Создание объекта Ticket

Префикс услуги + номер = "К-472"

4

Добавление в очередь

commonQueue.add(t) — клиент в ожидании

Made with GAMMA

ЛОГИКА ОБСЛУЖИВАНИЯ

Вызов клиента: callNextCustomer()

Алгоритм round-robin

Окна обслуживают по очереди — переменная `lastServedWindow` хранит индекс последнего активного окна:

```
lastServedWindow =
  (lastServedWindow + 1) % 5;
int window = lastServedWindow + 1;
```

⚠ ⚠️ С вероятностью 30% окно «занято документами» — ход пропускается, талон остаётся в очереди.

Made with GAMMA

АНАЛИТИКА

Статистика: showFullStats()

Что показывает отчёт?

- Общее число клиентов в очереди
- Разбивка по категориям услуг
- Номер последнего активного окна

Для подсчёта используется `EnumMap<Service, Integer>` — компактная и быстрая структура для enum-ключей.

EnumMap итерируется в порядке объявления enum — вывод всегда предсказуем.

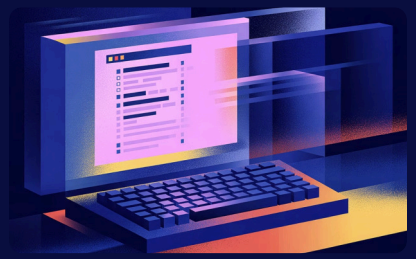
```
Map<Service, Integer> counts =
 new EnumMap<>(Service.class);

for (Ticket t : commonQueue)
 counts.put(t.service,
 counts.getOrDefault(
 t.service, 0) + 1);
```

Made with GAMMA

ИНТЕРФЕЙС

Консольный терминал



Меню клиента

Пункты 1–4: выбор услуги и получение талона. Пункт 0: выход из системы.

Пульт сотрудника

Пункт 5: вызов следующего клиента с анимацией «ДЗЫНЬ!» и указанием номера окна.

Отчёт системы

Пункт 6: полная статистика — размеры очередей по категориям и активное окно.

Made with GAMMA

Ключевые идеи проекта

Enum-типы

Строгая типизация услуг без магических строк

Queue<T>

FIFO-очередь через LinkedList — O(1) операции

Round-robin

Честное распределение нагрузки по 5 окнам

EnumMap

Эффективный подсчёт статистики по категориям

Проект демонстрирует практическое применение стандартных коллекций Java для моделирования реального бизнес-процесса.

Made with GAMMA